DOCUMENT 12

Learning System Guide

Pattern knowledge base, signal quality weighting, 3-gate CI evaluation, rever

NO GUESSWORK LIGHTING
v1.0 · March 2026 · Confidential

NGW

Overview

The NGW learning system is a closed-loop pipeline that turns production user outcomes into validated engine improvements. It combines a curated pattern knowledge base, quality-weighted signal aggregation, a 3-gate CI evaluator, and revenue projection to ensure every proposed change is safe, signal-sufficient, and worth shipping.

Nothing is auto-implemented without passing all three gates. Low-risk changes can auto-deploy once gates clear; medium risk requires human review; high risk always requires a human gate regardless of gate scores.

27	3-Gate CI	3 Risk tiers	+$2,063
Patterns in knowledge base	Signal · Benchmark · Pattern	low / medium / high	Annual uplift, S3 vs S1

Architecture — Six Modules

Module	Location	Role
ingestion	engine/learning/ingestion.py	Analytics failure clusters (6 failure modes)
auto_candidate	engine/learning/auto_candidate.py	Failure cluster typed candidate proposals
sandbox_eval	engine/learning/sandbox_eval.py	Gold Set evaluation — safe / risky / blocked
monitoring	engine/learning/monitoring.py	Post-release metric tracking at 7/14/30d windows

knowledge	engine/learning/knowledge.py	Pattern KB · signal weighting · MIN_SIGNALS gate
ci_gate	engine/learning/ci_gate.py	3-gate CI evaluator with risk-tier disposition
revenue	engine/learning/revenue.py	CVR delta revenue projection · 30-day simulation

Pattern Knowledge Base

engine/learning/knowledge.py holds a PATTERN_KNOWLEDGE_BASE dict with 27 entries. Each entry records the pattern's risk level, minimum signals for change, and known failure symptoms with fix steps. The risk level drives gate thresholds.

Risk Level	Min Weighted Signals	Patterns (examples)	Auto-Deploy?
low	25	broad, short, high_key, low_key	Yes, if gates pass
medium	75	split, butterfly, three_point, rim	Human review required
high	200	loop, rembrandt, clamshell, unknown	Human gate always

engine/learning/knowledge.py — PatternEntry

```
@dataclass
class PatternEntry:
    pattern_id:            str
    display_name:          str
    family:                str   # portrait|editorial|commercial|tabletop
    risk_level:            str   # low | medium | high
    min_signals_for_change: int   # MIN_SIGNALS[risk_level]
    symptoms:              List[SymptomEntry]
    tags:                  List[str]

# Access helpers
entry  = get_pattern_entry("clamshell")   # → PatternEntry
high   = list_high_risk_patterns()         # → ["loop","rembrandt","clamshell",....
thresh = get_min_signals("rembrandt")      # → 200
```

Pattern Comparison System

When analysis confidence falls below 0.72 and alternative patterns are present, the UI switches into comparison mode. The comparison system surfaces the two most likely patterns side-by-side so the photographer can confirm or reject the engine's top pick — adding a high-quality ambiguity signal back to the learning loop.

Source: ui/src/knowledge/graph.js builds a bidirectional confusionWith graph. ui/src/knowledge/index.js getPatternComparisons() returns structured comparison pairs including key distinguishing cues (e.g., nose shadow side, catch-light position, shadow falloff) for the detected and candidate patterns.

Component	Location	Role

buildGraph()	knowledge/graph.js	Computes patternToPatterns + patternToSymptoms maps once
getPatternComparisons()	knowledge/index.js	Returns ComparisonPair[] for a pattern and its confusionWith list
ResultPatternComparePrompt	screens/ResultsScreen.jsx	Rendered when showComparison === true (confidence < 0.72)
confusionWith	knowledge/patterns.js each entry	Array of pattern IDs most often confused with this one

ui/src/knowledge/index.js — comparison trigger logic

```
// In buildResultsData():
const showComparison = confidence < 0.72 && alternatives.length > 0;
const comparisons    = showComparison
  ? getPatternComparisons(patternId)
  : [];

// Each ComparisonPair includes:
// { patternA, patternB, distinguishers: [
//   { cue, aValue, bValue, weight },   // e.g. nose shadow position
//   ...] }
```

WHY COMPARISON SIGNALS ARE HIGH-VALUE

• A photographer who manually selects "Loop — not Rembrandt" after seeing ambiguous results

• provides a human-verified ambiguity correction. These signals carry source="live" + outcome

• "failed" weight in the knowledge system. Accumulating 15–20 such corrections reliably shifts

• the classifier thresholds for the confused pattern pair.

Signal Quality Weighting

Raw session signals are weighted before counting toward a gate threshold. A pro photographer's expert-reviewed correction carries 5× the weight of a beginner's live session. This prevents a flood of low-quality signals from triggering unsafe changes.

weight = tier × source × outcome × confidence_boost (if confidence ≥ 0.75)

Factor	Value	Multiplier
Tier	pro	2.0×
	advanced	1.5×
	intermediate	1.0×
	beginner	0.5×
	unknown	0.7×
Source	expert_review	2.5×
	internal	1.2×
	live	1.0×
	seeded	0.4×
Outcome	nailed_it / failed	1.0×
	close	0.7×
	unknown	0.3×
Conf boost	≥ 0.75	+1.8× (capped at 5.0 total)

engine/learning/knowledge.py — compute_signal_weight()

```
w = compute_signal_weight(
    skill_tier="pro",
    confidence_score=0.9,
    source="expert_review",
    outcome="nailed_it",
)  # → 5.0  (capped)

w = compute_signal_weight("intermediate", 0.8, "live", "failed")
# → 1.8  (confidence boost applied)

w = compute_signal_weight("beginner", 0.3, "live", "unknown")
# → 0.15
```

3-Gate CI Evaluator

Before any candidate can be promoted, it must pass three gates in sequence. Failing any gate blocks promotion entirely — the gates cannot be overridden except by the override_blocked flag with an explicit written reason.

Gate	What it checks	Failure condition
1 Signal Sufficiency	Weighted signals ≥ MIN_SIGNALS[risk_level]	Returns disposition=insufficient
2 Benchmark Delta	Overall score drop ≤ 2%	Returns disposition=blocked
3 Pattern Regression	No single pattern drops > 5%	Returns disposition=blocked

If all gates pass, the risk level determines the final disposition:

Risk Level	Disposition	What happens next

low	auto_deploy	Change deployed automatically; monitoring window opens
medium	human_review	Curator notified; must approve before deploy
high	human_gate	Committee review required regardless of gate scores
any	blocked	Gate 2 or 3 failed; fix regressions before re-evaluating
any	insufficient	Gate 1 failed; accumulate more signals and retry

engine/learning/ci_gate.py — evaluate_candidate_dict()

```
# Pure-function test (no DB)
result = evaluate_candidate_dict(
    candidate       = {"id": "c1", "pattern_id": "loop", "risk_level": "low"},
    insight         = aggregate_signals_for_pattern("loop", signals),
    benchmark_delta = 0.01,   # +1% overall score
)
# result.disposition → "auto_deploy"
# result.overall_verdict → "pass"

# DB-backed evaluation (loads signals + runs benchmark)
result = evaluate_candidate_gate("cand-uuid-here")
print(summarise_gate_result(result))
# → "■ Auto-deploy: 3/3 gates passed (low risk)"
```

Failure Detection & Candidate Pipeline

Failure Mode	Candidate Type	Trigger Condition
conversion_gap	blueprint_correction	CVR < 1.0% with n ≥ 5 sessions
confidence_mismatch	confidence_recalibration	CVR > 2σ below fleet mean
step_deviation	shoot_mode_step_fix	Global match rate < 30%

pattern_drift	dataset_promotion	Daily volume decline > 40%
trust_gap	trust_safety	Matched sessions convert worse than unmatched
env_conversion_gap	blueprint_correction	Environment-segmented CVR gap

CANDIDATE SAFETY RULES

- Candidates always begin in status="proposed". No candidate is ever auto-accepted.
- sandbox_eval verdict "blocked" prevents promotion. Override requires override_blocked=true + written reason.
- Confidence_recalibration is the only candidate type with an auto-apply endpoint.
- All other types require manual engine edits and a benchmark re-run.

Benchmark Enforcement System

The benchmark enforcement system ensures no candidate can degrade core accuracy. Gate 2 of the CI evaluator runs sandbox_eval.py against the Gold Set before any candidate can advance. Gate 3 adds a per-pattern regression check. Together they create a two-layer safety net: fleet-level accuracy (Gate 2) and individual pattern regression (Gate 3).

Gate	Enforcement layer	Pass threshold	Source module
Gate 2	Benchmark delta	Overall acc Δ ≥ −2%	sandbox_eval.py vs Gold Set

Gate 3	Pattern regression	Per-pattern acc Δ ≥ −5%	per-pattern subset of Gold Set
Blocked	Both gates failed	N/A — manual override req	ci_gate.py disposition=blocked

Gold Set promotion: when a session reaches confidence ≥ 0.85 with outcome "nailed_it" it becomes a gold set candidate (GET /lab/signals/gold-set-suggestions). Human-reviewed gold set images set the accuracy baseline that Gates 2 and 3 measure against. The larger and more diverse the gold set, the tighter the enforcement.

engine/learning/ci_gate.py — gate dispositions

```
# Disposition map after all gates:
# "auto_deploy"    → all 3 passed + risk_level="low"
# "human_review"   → all 3 passed + risk_level="medium"
# "human_gate"     → all 3 passed + risk_level="high"
# "blocked"        → Gate 2 or Gate 3 failed
# "insufficient"   → Gate 1 (signal count) failed

# Run it programmatically:
result = evaluate_candidate_dict({
    "pattern_id":          "rembrandt",
    "weighted_signals":    212,
    "benchmark_delta":     -0.01,
    "pattern_acc_delta":   0.02,
    "risk_level":          "high",
})
# → CIGateResult(disposition="human_gate", gates_passed=[1,2,3])
```

ADDING BENCHMARK CASES

- POST /lab/benchmarks/cases — add individual cases manually.
- POST /lab/benchmarks/from-gold-set — promote gold set suggestions in bulk.
- Run python3 scripts/run_benchmarks.py after each engine edit.
- Zero FAIL results required before any Gate 2 benchmark delta is computed.

Revenue Projection & 30-Day Simulation

engine/learning/revenue.py quantifies the revenue impact of a CVR improvement. It answers three questions: (1) How much revenue does fixing this pattern unlock? (2) Which day does the signal gate clear? (3) What's the difference between treating all patterns identically vs. risk-tiering the deployment schedule?

engine/learning/revenue.py — compute_revenue_impact()

```
scenario = compute_revenue_impact(
    pattern_id       = "rembrandt",
    sessions_per_day = 40,
    before_cvr       = 0.042,   # 4.2% current
    after_cvr        = 0.065,   # 6.5% target
    arpu             = 9.00,    # $9 per conversion
)
# scenario.cvr_lift                  → +2.3pp
# scenario.additional_conversions_30d → 27.6
# scenario.delta_revenue_30d          → $248.40
# scenario.annualised_delta           → $2,980.80
```

project_30_day_metrics() simulates day-by-day signal accumulation for a list of patterns under a given scenario. For each pattern it computes:

Field	Meaning
gate_unlock_day	Day signals cross MIN_SIGNALS[risk_level] threshold
deploy_day	gate_unlock_day + risk deploy lag (low=0d, med=+3d, high=+7d)
cumulative_revenue_delta	Running Σ of daily revenue delta from deploy_day onward
day_snapshots	List of 30 DaySnapshot objects — one per day, suitable for charts

scripts/simulate_30day.py runs the full simulation across three strategic scenarios with ASCII day-by-day signal bars, gate unlock events, and a final comparison table. The CRITICAL insight: risk-tiered deployment (Scenario 3) outperforms treating all patterns as high-risk by $172/month (+$2,063/year) — while maintaining the same safety gates for high-risk patterns.

bash — run the 30-day simulation

```bash
# Full 3-scenario comparison (recommended)
python3 scripts/simulate_30day.py

# Single scenario (Controlled Autonomy)
python3 scripts/simulate_30day.py --scenario 3

# JSON output for API / dashboard integration
python3 scripts/simulate_30day.py --json --scenario 3

# Example Scenario 3 output (key events):
# Day  1: Loop gate cleared  (low risk, 25 signals) → deploy same day → +$224
# Day  5: Clamshell gate cleared (med risk, 75 sig) → deploy day 8   → +$296
# Day 15: Rembrandt gate cleared (high risk, 200 sig)→ deploy day 22 → +$99
# ██████████████████████████████████████████████████████████████████████
# 30d total: +$620.01   |   Annualised: +$7,440.12
```

Scenario	Loop	Clamshell	Rembrandt	30d Delta	Annualised	vs S1
1 — Conservative	High day 10	High day 10	High day 10	+$448	+$5,377	baseline
2 — Moderate	Low day 1	High day 10	Med day 5	+$600	+$7,204	+$1,827/yr
3 — Controlled Auto	Low day 1	Med day 5	High day 15	+$620	+$7,440	+$2,063/yr

THE RISK-TIER ADVANTAGE

- Scenario 3 dominates because Loop and Clamshell are the highest-volume patterns.
- Deploying Loop on Day 1 (not Day 10) captures 9 extra days of revenue lift.
- Rembrandt is correctly delayed — it is a high-risk ambiguous pattern requiring 200 signals.
- The $172/month difference vs Scenario 1 compounds to $2,063/year at current volume.
- This justifies maintaining the knowledge base risk tiers with discipline.

API integration: POST /lab/learning/revenue/simulate accepts a JSON array of SimulateScenario objects and returns a RevenueProjection array with full day_snapshots suitable for rendering as a 30-day bar chart in the Lab UI.

API Endpoints — Knowledge & Revenue

Method	Path	Purpose
GET	/lab/learning/knowledge	Full pattern knowledge base (27 patterns)
GET	/lab/learning/knowledge/{id}	Single pattern entry with symptoms + fix steps
POST	/lab/learning/knowledge/{id}/signals	Aggregate weighted signals for a pattern
POST	/lab/learning/knowledge/{id}/ci-gate	Run full 3-gate CI evaluation for a candidate
POST	/lab/learning/revenue/impact	Compute CVR delta revenue impact
POST	/lab/learning/revenue/simulate	30-day simulation for N scenario configs

AUTH REQUIREMENTS

- All /lab/learning/* endpoints require JWT with email in NGW_DEV_EMAILS.
- Set NGW_DEV_MODE=1 to bypass auth locally. Never in production.
- Signal aggregate endpoint reads db/signals.py — ensure ngw.db is seeded for local testing.

Monitoring — Post-Release Windows

After a candidate is released, monitoring.py captures metric snapshots at 7, 14, and 30 days. Two alert types trigger automatic notifications:

Alert Type	Trigger Condition	Recommended Action

candidate_regression	success_rate_delta < –5pp OR conf < –0.1	Review candidate; consider rollback
rollback_review	CVR delta < –3pp OR trust delta < –0.15	Immediate rollback review required

bash — monitoring sweep

```
# Sweep all 3 windows (7d, 14d, 30d)
POST /lab/learning/monitoring/sweep-all

# Check alerts
GET  /lab/learning/monitoring

# Detailed report for one release
GET  /lab/learning/monitoring/{attribution_id}
```